

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине: «Технологии и методы программирования»

на тему: «Разработка защищённого REST API и SPA»

Вариант 21: «Безопасность в спортивных сооружениях и на мероприятиях»

Выполнил:

Студент гр. «АБс-426», «АВТФ»,

Терещенко Никита Сергеевич

«___» _____ 2026 г.

(подпись)

Проверили:

Ассистент кафедры ЗИ

Питько Яна Андреевна

Старший преподаватель кафедры ЗИ

Медведев Михаил Александрович

«___» _____ 2026 г.

(подпись)

Новосибирск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
Цель и задачи	4
ХОД ВЫПОЛНЕНИЯ	4
Проектирование базы данных	4
REST API	4
Аутентификация и авторизация	5
Интерфейс и роли	7
Безопасность и обработка ошибок	9
CI/CD и тестирование	9
Ответы на контрольные вопросы	9
ЗАКЛЮЧЕНИЕ	10
ПРИЛОЖЕНИЕ А. ЛИСТИНГ МЕХАНИЗМОВ ЗАЩИТЫ.....	11
Контроллер аутентификации	11
Middleware JWT/RBAC	12
Маршрутизация SPA	12

ВВЕДЕНИЕ

Лабораторная работа развивает SPA учёта инцидентов до защищённой информационной системы с PostgreSQL, регистрацией, JWT, bcrypt и серверной RBAC. Объектом являются инциденты на стадионах, в спортивных комплексах и на массовых мероприятиях.

Цель и задачи

спроектировать базу данных;
реализовать REST API и корректные статусы;
добавить регистрацию, JWT и роли;
создать SPA с защищёнными маршрутами;
реализовать пагинацию, фильтры и ошибки;
настроить CI и выполнить серверную проверку.

ХОД ВЫПОЛНЕНИЯ

Проектирование базы данных

Таблица 1 – Сущности БД

Таблица	Поля	Назначение
users	id, login, password_hash, role	учётные записи и роли
incidents	id, title, category, zone, threat_level, status, operator_name, description, created_at	карточки происшествий

Логин уникален, роль ограничена CHECK, обязательные атрибуты имеют NOT NULL. Пароль отсутствует в открытом виде: сохраняется только bcrypt-хеш cost 12.

REST API

Таблица 2 – Основные эндпоинты

Метод	URI	Доступ	Назначение
POST	/api/auth/register	открытый	регистрация оператора
POST	/api/auth/login	открытый	JWT
GET	/api/incidents	все	список
POST	/api/incidents	admin	создание
PATCH	/api/incidents/:id/status	operator/admin	статус

PUT/DELETE	/api/incidents/:id	admin	изменение/удаление
GET/POST/PUT/DELETE	/api/users	admin	пользователи

Аутентификация и авторизация

После `bcrypt.compare` сервер подписывает JWT секретом из окружения. Claims `sub`, `login` и `role` действуют два часа и имеют issuer `sport-security-api`. Middleware отклоняет отсутствующий, изменённый или просроченный токен кодом 401, а недостаточную роль — 403. Заголовок `x-user-role` не принимается как доказательство роли.

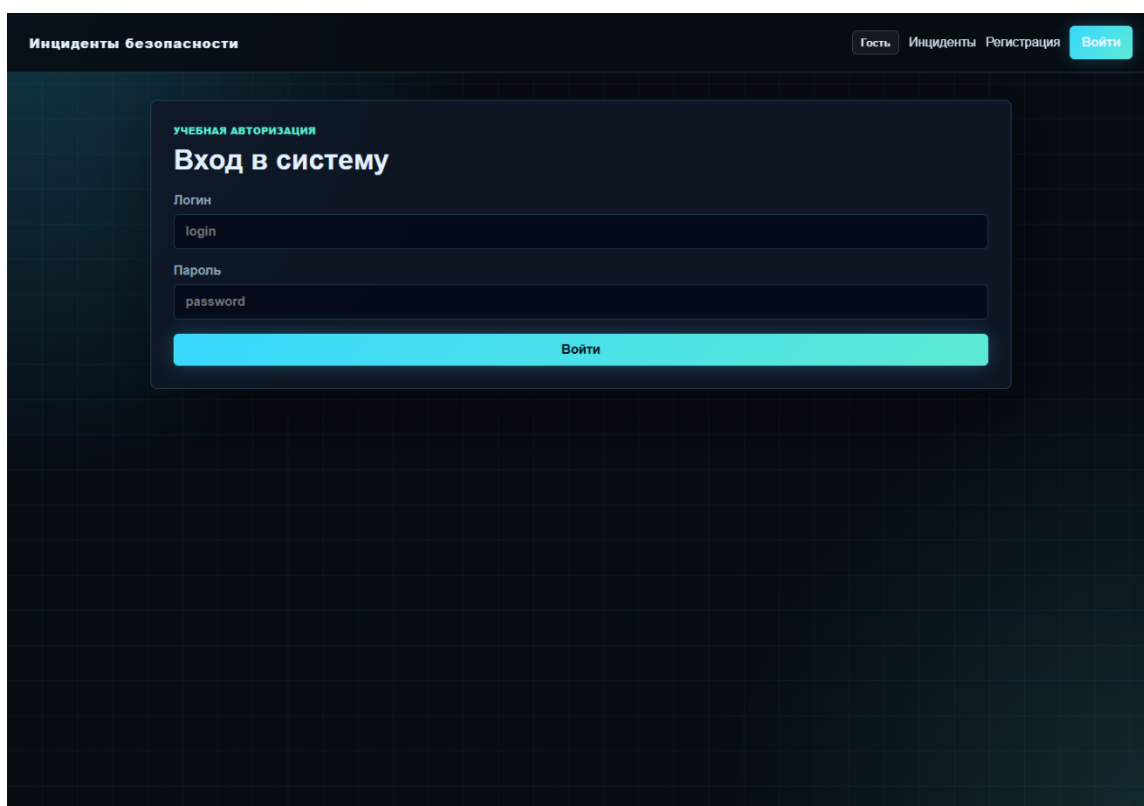


Рисунок 1 – Вход в защищённое приложение

The screenshot shows a web application interface with a dark background and a grid pattern. At the top, there is a header bar with the text 'Инциденты безопасности' (Security Incidents) on the left and a navigation menu on the right containing 'admin (Администратор)', 'Инциденты', 'Добавить инцидент' (highlighted in red), 'Пользователи', and 'Выйти'. The main content area features a central form titled 'Регистрация оператора' (Operator Registration) with the subtitle 'новая учётная запись' (new account). The form contains three input fields: 'Логин' (Login), 'Пароль' (Password), and 'Повторите пароль' (Repeat password). Below these fields is a red button labeled 'Зарегистрироваться' (Register).

Рисунок 2 – Регистрация оператора

The screenshot shows the same web application interface. The main content area now displays a section titled 'Инциденты безопасности' (Security Incidents) with the subtitle 'SPA / REST API / POSTGRESQL'. Below the title, there are three filters: a text input field labeled 'Поиск по названию, зоне или категории' (Search by name, zone or category), a dropdown menu labeled 'Все статусы' (All statuses), and another dropdown menu labeled 'Все уровни угрозы' (All threat levels). Below the filters is a red button labeled 'Загрузка...' (Loading...).

Рисунок 3 – Список и фильтры

Интерфейс и роли

Гость просматривает открытые данные; operator меняет статус; admin выполняет полный CRUD и управляет пользователями. ProtectedRoute скрывает недоступные страницы, но решающее ограничение всегда выполняется backend.

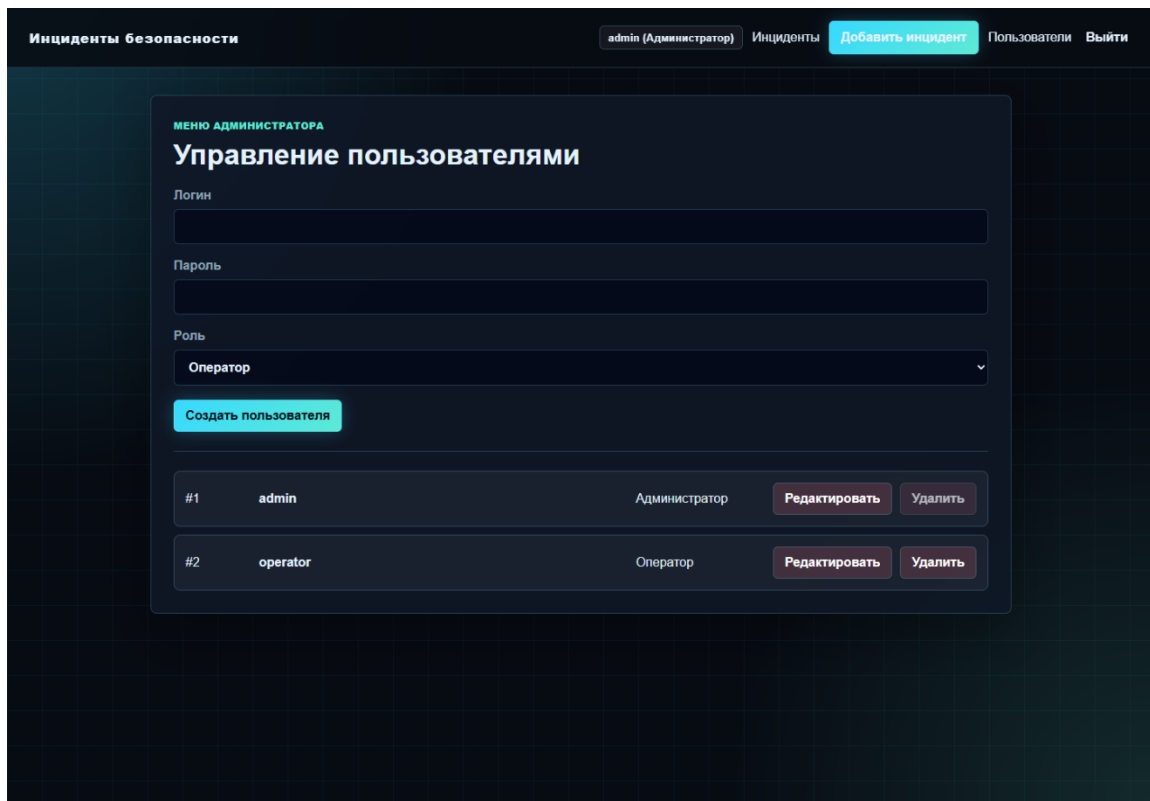


Рисунок 4 – Управление пользователями

Инциденты безопасности

admin (Администратор)ИнцидентыДобавить инцидентПользователиВыйти

создание

Добавить инцидент

Название

Категория

Выберите категорию

Зона

Уровень угрозы

Выберите уровень

Статус

Выберите статус

Оператор

Описание

СохранитьОтмена

Рисунок 5 – Добавление инцидента

Инциденты безопасности

admin (Администратор)ИнцидентыДобавить инцидентПользователиВыйти

инцидент #1

Несанкционированный проход в VIP-зону

Категория: Контроль доступаЗона: VIP-зона

Уровень угрозы: ВысокийСтатус: Открыт

Оператор: ИвановСоздан: 21.08.2026, 11:39:19

Посетитель пытался пройти через служебный вход без пропуска.

НазадРедактировать

Рисунок 6 – Детальная карточка

Безопасность и обработка ошибок

Helmet устанавливает security headers, CORS ограничивается переменной окружения, JSON ограничен 64 КБ. SQL использует параметры. Сообщения 4xx не раскрывают стек. В production приложение должно работать за HTTPS reverse proxy; refresh token следует хранить в HttpOnly cookie.

CI/CD и тестирование

Frontend успешно собран. Smoke-тест на <http://m4nuul.ru:5054> подтвердил вход двух ролей, запрет поддельной роли и admin-действия оператором, CRUD, изменение статуса, удаление и пагинацию. PostgreSQL tipm_lab4 изолирована от других работ. Исходный код: <https://github.com/M4NUUL/laboratorywork4/tree/ready-variant-21>.

Ответы на контрольные вопросы

1. Что такое REST?

REST — архитектурный стиль, в котором ресурсы адресуются URI, запросы не хранят состояние сессии на сервере и используют единый HTTP-интерфейс.

2. Что такое JWT?

JWT — подписанный набор claims. Сервер проверяет подпись, issuer и срок действия перед доступом к защищённому маршруту.

3. Аутентификация и авторизация — в чём разница?

Аутентификация устанавливает личность, авторизация проверяет разрешённость конкретного действия.

4. Почему используется bcrypt?

bcrypt является адаптивной функцией хеширования паролей; cost 12 делает массовый перебор существенно дороже.

5. Как реализована RBAC?

Middleware извлекает роль из проверенного JWT и допускает запрос только для operator/admin согласно маршруту.

6. Где хранится токен?

Учебный клиент использует localStorage. В production предпочтителен короткоживущий access token и refresh token в HttpOnly Secure SameSite cookie.

7. Как снижены XSS и CSRF?

React экранирует текст, Helmet задаёт CSP и защитные заголовки; Bearer token не отправляется браузером автоматически, что снижает CSRF-риск.

8. Как организованы пагинация и фильтрация?

Клиент передаёт limit, offset и фильтры; сервер строит параметризованный запрос и возвращает ограниченную выборку.

9. Что делает CI/CD?

Workflow устанавливает зависимости, запускает проверки и собирает SPA при push; развёртывание выполняется только после успешного этапа.

10. Как обрабатываются 401 и 403?

401 очищает недействительную локальную сессию и переводит на вход; 403 отображает отказ без раскрытия внутренних деталей.

ЗАКЛЮЧЕНИЕ

Создано полноценное защищённое веб-приложение, объединяющее React SPA, Express API и PostgreSQL. В отличие от первой лабораторной реализованы регистрация, bcrypt, JWT с ограниченным сроком, серверная RBAC, управление пользователями, защитные заголовки и CI. Фактический smoke-тест подтвердил не только позитивный CRUD, но и отказы 401/403. Архитектура допускает расширение новыми сущностями и ролями. Для итоговой РГР следующий этап добавляет аудит, формальный OpenAPI-контракт, резервное копирование и эксплуатационные процедуры.

ПРИЛОЖЕНИЕ А. ЛИСТИНГ МЕХАНИЗМОВ ЗАЩИТЫ

Контроллер аутентификации

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const db = require('../db');

const issueToken = (user) => jwt.sign(
  { sub: user.id, login: user.login, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '2h', issuer: 'sport-security-api' }
);

const login = async (req, res) => {
  const loginValue = String(req.body.login || "").trim();
  const password = String(req.body.password || "");
  if (!loginValue || !password) return res.status(400).json({ error: 'Введите логин и пароль' });
  try {
    const { rows } = await db.query('SELECT id, login, password_hash, role FROM users WHERE login = $1', [loginValue]);
    if (!rows[0] || !(await bcrypt.compare(password, rows[0].password_hash))) {
      return res.status(401).json({ error: 'Неверные учётные данные' });
    }
    const { password_hash, ...user } = rows[0];
    res.json({ ...user, token: issueToken(user) });
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
};

const register = async (req, res) => {
  const loginValue = String(req.body.login || "").trim();
  const password = String(req.body.password || "");
  if (!/^[a-zA-Z0-9_-]{3,40}$/.test(loginValue) || password.length < 8) {
    return res.status(400).json({ error: 'Логин: 3–40 символов; пароль: минимум 8 символов' });
  }
  try {
    const passwordHash = await bcrypt.hash(password, 12);
    const { rows } = await db.query(
      `INSERT INTO users (login, password_hash, role) VALUES ($1, $2, 'operator') RETURNING id, login, role`,
      [loginValue, passwordHash]
    );
  }
};
```

```

    );
    res.status(201).json({ ...rows[0], token: issueToken(rows[0]) });
  } catch (error) {
    if (error.code === '23505') return res.status(409).json({ error: 'Логин уже занят'
});
    console.error(error);
    res.status(500).json({ error: 'Ошибка сервера' });
  }
};

```

```

module.exports = { login, register };

```

Middleware JWT/RBAC

```

const jwt = require('jsonwebtoken');

```

```

const getBearerToken = (req) => {
  const [scheme, token] = (req.headers.authorization || '').split(' ');
  return scheme === 'Bearer' ? token : '';
};

```

```

const authenticate = (req, res, next) => {
  const token = getBearerToken(req);
  if (!token) return res.status(401).json({ error: 'Требуется вход в систему' });
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET, { issuer: 'sport-
security-api' });
    req.user = { id: Number(payload.sub), login: payload.login, role: payload.role };
    next();
  } catch {
    res.status(401).json({ error: 'Токен недействителен или просрочен' });
  }
};

```

```

const allowRoles = (...roles) => (req, res, next) => authenticate(req, res, () => {
  if (!roles.includes(req.user.role)) return res.status(403).json({ error:
'Недостаточно прав' });
  next();
});

```

```

module.exports = { authenticate, adminOnly: allowRoles('admin'),
operatorOrAdmin: allowRoles('operator', 'admin') };

```

Маршрутизация SPA

```

import { BrowserRouter, Navigate, Route, Routes } from 'react-router-dom';
import './App.css';

```

```

import Navbar from './components/Navbar';
import ProtectedRoute from './components/ProtectedRoute';
import LoginPage from './pages/LoginPage';
import RegisterPage from './pages/RegisterPage';
import IncidentsPage from './pages/IncidentsPage';
import IncidentDetailPage from './pages/IncidentDetailPage';
import IncidentFormPage from './pages/IncidentFormPage';
import UsersPage from './pages/UsersPage';

function App() {
  return (
    <BrowserRouter future={{ v7_startTransition: true, v7_relativeSplatPath: true
}}>
      <div className="app-layout">
        <Navbar />
        <main className="app-content">
          <Routes>
            <Route path="/login" element={<LoginPage />} />
            <Route path="/register" element={<RegisterPage />} />
            <Route path="/" element={<IncidentsPage />} />
            <Route path="/detail/:id" element={<IncidentDetailPage />} />
            <Route
              path="/add"
              element={{
                <ProtectedRoute allowedRoles={['admin']>
                  <IncidentFormPage />
                </ProtectedRoute>
              }}
            />
            <Route
              path="/edit/:id"
              element={{
                <ProtectedRoute allowedRoles={['admin']>
                  <IncidentFormPage />
                </ProtectedRoute>
              }}
            />
            <Route
              path="/admin/users"
              element={{
                <ProtectedRoute allowedRoles={['admin']>
                  <UsersPage />
                </ProtectedRoute>
              }}
            />
          </Routes>
        </main>
      </div>
    </BrowserRouter>
  );
}

```

```
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </main>
  </div>
</BrowserRouter>
);
}

export default App;
```